

Concurrency and Distributed Systems

Week 4 – Part 4: AtomicInteger, CAS, Lock-Free Programming, and Contention

Dr. Mohamad Aoude

Faculty of Engineering

May 11, 2026

Part 4 Roadmap

- 1 Why We Need Another Tool
- 2 AtomicInteger
- 3 Compare-And-Swap (CAS)
- 4 Lock-Free Programming Intuition
- 5 Performance and Contention
- 6 Comparison with Locks
- 7 Lab Preparation and Week Summary

Where We Are in Week 4

Part 1

We learned why threads can observe stale values.

Part 2

We learned that `volatile` provides:

- visibility
- ordering
- happens-before through volatile communication

Part 3

We learned that visibility is not enough:

- `counter++` is not atomic
- lost updates happen

Learning Objectives

By the end of this lecture, students should be able to:

- explain why `AtomicInteger` solves a different problem than `volatile`
- use `AtomicInteger` for correct atomic increments
- explain the idea of compare-and-swap (CAS)
- understand retry-based optimistic update logic
- define lock-free programming at an introductory level
- compare atomic variables with locks from Week 3
- reason about performance under contention

Why We Need Another Tool

After Part 3, students should already know:

- a plain shared counter is unsafe
- a volatile counter is still unsafe
- locks can make the counter safe

Natural next question

Can we get a correct atomic increment without surrounding every update with a traditional lock?

Answer

Yes, for many cases Java provides atomic classes such as:

`AtomicInteger`

The Basic AtomicInteger Example

```
1      import java.util.concurrent.atomic.AtomicInteger;
2
3      class AtomicCounterDemo {
4          static AtomicInteger counter = new AtomicInteger(0);
5
6          static void increment() {
7              counter.incrementAndGet();
8          }
9      }
```

Why this matters

This performs an atomic read-modify-write update on the shared value.

Important

This is not just “a visible int.” It is an object that provides atomic operations.

AtomicInteger vs volatile int

Mechanism	Visibility	Ordering	Atomic Increment
<code>volatile int</code>	Yes	Yes	No
<code>AtomicInteger</code>	Yes	Yes	Yes

Interpretation

`AtomicInteger` does not merely expose a value. It exposes operations with stronger guarantees.

A Complete Atomic Counter Example

```
1      import java.util.concurrent.atomic.AtomicInteger;
2
3      public class AtomicCounterDemo {
4          static AtomicInteger counter = new AtomicInteger(0);
5
6          public static void main(String[] args) throws
              Exception {
7              Thread t1 = new Thread(() -> {
8                  for (int i = 0; i < 100000; i++) {
9                      counter.incrementAndGet();
10                 }
11             });
12
13             Thread t2 = new Thread(() -> {
14                 for (int i = 0; i < 100000; i++) {
15                     counter.incrementAndGet();
16                 }
17             });
```


What Makes This Work?

Key idea

`AtomicInteger` provides operations that behave atomically with respect to concurrent updates.

So for:

```
counter.incrementAndGet()
```

we do *not* get the unsafe pattern:

- read shared value
- compute locally
- race during write-back

Instead

The update is executed using a lower-level atomic mechanism.

Enter Compare-And-Swap (CAS)

CAS idea

Compare-And-Swap is a hardware-supported atomic primitive used to update memory conditionally.

It takes:

- a memory location
- an expected old value
- a new value

Operation:

- if the current value equals the expected value, update succeeds
- otherwise update fails

This is the engine behind many atomic classes

`AtomicInteger` is conceptually built on this kind of mechanism.

CAS in Pseudo-Logic

Pseudo-logic

```
if (currentValue == expectedValue) {  
    currentValue = newValue;  
    return success;  
} else {  
    return failure;  
}
```

Important

The whole compare-and-update is one atomic hardware-supported step.

Why this matters

Two threads cannot both “win” with the same expected old value when racing on the same

The compareAndSet Operation in Java

```
1      import java.util.concurrent.atomic.AtomicInteger;  
2  
3      AtomicInteger value = new AtomicInteger(10);  
4  
5      boolean success = value.compareAndSet(10, 11);
```

Meaning:

- if the current value is exactly 10, replace it with 11
- otherwise do nothing and report failure

Interpretation

This is optimistic concurrency:

"I assume the value is still what I last saw. If not, I will retry."

How Atomic Increment Can Be Built from CAS

Conceptually, an atomic increment can be written as:

```
1      while (true) {  
2          int oldValue = counter.get();  
3          int newValue = oldValue + 1;  
4          if (counter.compareAndSet(oldValue, newValue)) {  
5              break;  
6          }  
7      }
```

Interpretation

The thread:

- reads the current value
- computes a candidate new value
- tries to install it atomically

Why the Retry Is Necessary

Suppose two threads both read:

$$oldValue = 5$$

Both want to install:

$$newValue = 6$$

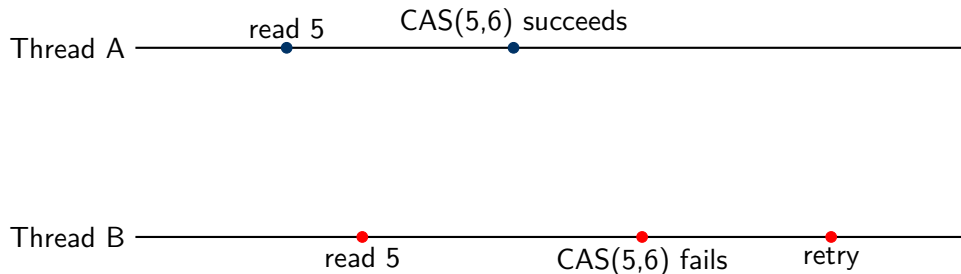
With CAS:

- only one thread succeeds in changing 5 to 6
- the other thread fails because the current value is no longer 5
- the failing thread must re-read and try again

Key point

Failure is not a bug here. It is part of the normal optimistic update strategy.

CAS Timeline



Outcome

Only one update wins per expected state. The loser retries with the new observed value.

What “Lock-Free” Means Here

Introductory definition

A lock-free algorithm avoids traditional blocking locks and instead relies on atomic primitives such as CAS.

Practical classroom meaning

Threads do not wait to acquire a monitor or explicit lock for the update. They attempt the update optimistically and retry if needed.

Be careful

Lock-free does *not* mean:

- no contention
- no retries
- always faster

Why Lock-Free Sounds Attractive

Lock-free approaches can offer benefits such as:

- no traditional lock acquisition on simple operations
- avoidance of some blocking overheads
- good scalability for some workloads
- reduced risk of classic lock-based waiting chains

Typical good use

Shared counters, simple flags with atomic transitions, and lightweight concurrent state updates.

But Lock-Free Also Has Costs

Lock-free is not magic.

Possible costs:

- repeated retries under contention
- wasted work when CAS fails repeatedly
- cache line bouncing across cores
- more subtle reasoning than plain sequential code

Engineering reality

Under high contention, many threads may repeatedly fight over the same memory location.

Contention: The Real Performance Question

Contention

Contention occurs when many threads repeatedly access and update the same shared resource.

For a shared counter, this means:

- multiple threads compete for one value
- multiple CAS attempts fail
- synchronization traffic increases

Key lesson

Concurrency performance is not about ideology. It is about workload, contention level, and access pattern.

Low Contention vs High Contention

Low contention

- few collisions
- CAS often succeeds quickly
- atomics may perform very well

High contention

- many collisions
- repeated CAS failures
- retries waste CPU work

Important

The same mechanism can look excellent in one scenario and poor in another.

Comparison with Traditional Locks

synchronized / ReentrantLock	serialize access to a critical section
AtomicInteger	provides atomic operations on a value without a traditional lock block
Lock-based up- date	wait / enter / execute / release
CAS-based up- date	try / fail-or-succeed / retry if needed

Interpretation

Locks and atomics solve overlapping but not identical problems.

When Locks Are Still Better

Even after learning atomics, students should not think:

`CAS solves everything`

Locks are still often better when:

- several variables must be updated together
- a larger critical section must remain consistent
- code clarity matters more than micro-optimization
- the logic is complex and retry-based reasoning becomes messy

Important

Atomic classes are excellent for specific patterns, not for every concurrency design.

When AtomicInteger Is a Great Fit

AtomicInteger is especially good for:

- counters
- sequence numbers
- simple state transitions
- lightweight shared metrics

Typical idea

One shared variable, one atomic operation, many concurrent updates.

Optional Extension: LongAdder

For strong students, you may mention:

LongAdder

Why it exists

Under very high contention, repeatedly updating one single atomic value can become a bottleneck.

General idea

Instead of forcing all threads to hit one shared point, work can be spread across multiple internal cells and combined later.

This is an optimization topic, not the main core of Part 4.

A Benchmark Mindset

Students should compare at least four counter strategies:

- ① plain int
- ② volatile int
- ③ synchronized counter
- ④ AtomicInteger

For each version, measure:

- correctness of final result
- time under 2 threads
- time under 4, 8, or more threads

Pedagogical benefit

Students stop arguing abstractly and start observing real tradeoffs.

Counter Strategy Comparison

Strategy	Correct?	Blocks?	Good for
Plain int	No	No	nothing
volatile int	No for increment	No	concurrent visibility-only communication
synchronized counter	Yes	Yes	critical sections
AtomicInteger	Yes	No traditional lock	atomic value updates

What Students Must Not Conclude

They must **not** leave this lecture believing:

- “lock-free always means faster”
- “CAS never fails”
- “AtomicInteger replaces locks in every design”
- “retry is free”

Replace with

Atomics are powerful for specific update patterns, but performance and suitability depend on contention and design structure.

Part 4 Concept Summary

AtomicInteger	atomic shared-value operations in Java
CAS	compare current value and swap only if expectation still matches
Optimistic update	try update first, retry if interference occurred
Lock-free idea	avoid traditional blocking locks for some operations
Contention	many threads competing for the same shared state
Engineering lesson	no tool is universally best

Lab Preparation for Part 4

Lab 4.8 — Atomic Counter

Students replace an unsafe counter with `AtomicInteger` and verify correctness.

Lab 4.9 — CAS Retry Demo

Students manually code a retry loop using:

- `get()`
- `compareAndSet()`

Lab 4.10 — Contention Benchmark

Students compare:

- plain int
- volatile int
- synchronized counter

Suggested Discussion Questions

- 1 Why is `AtomicInteger` different from `volatile int`?
- 2 What does CAS actually check?
- 3 Why must a failed CAS retry?
- 4 What does lock-free mean in practical terms?
- 5 Why might atomics perform poorly under high contention?
- 6 When would a lock still be the clearer or safer solution?

Week 4 Final Synthesis

Part 1

why threads can observe stale memory

Part 2

how `volatile` provides visibility and ordering

Part 3

why visibility is not enough; atomicity and DCL

Part 4

how atomics and CAS support lock-free updates

The real progression

- 1 Threads can disagree about memory
- 2 Visibility can be restored
- 3 Visibility is not atomicity
- 4 Atomic updates can be built with CAS

Final Takeaway of the Whole Week

Week 4 takeaway

Concurrent correctness depends on three different questions:

- What can threads see?
- In what order can they see it?
- Does an update happen as one indivisible action?

Tool map

- `volatile` helps with visibility and ordering
- locks help with mutual exclusion and grouped atomicity
- atomic classes help with lock-free atomic updates

Questions?